

RosettaDash: Components that Travel

RosettaDash is a portmanteau of Rosetta, the Egyptian stone with Demotic, Greek, and hieroglyphic characters that enabled modern translation of extinct languages, and the modern web dashboard. The motivation behind it is to enable component creation that can be used for whichever framework you choose.

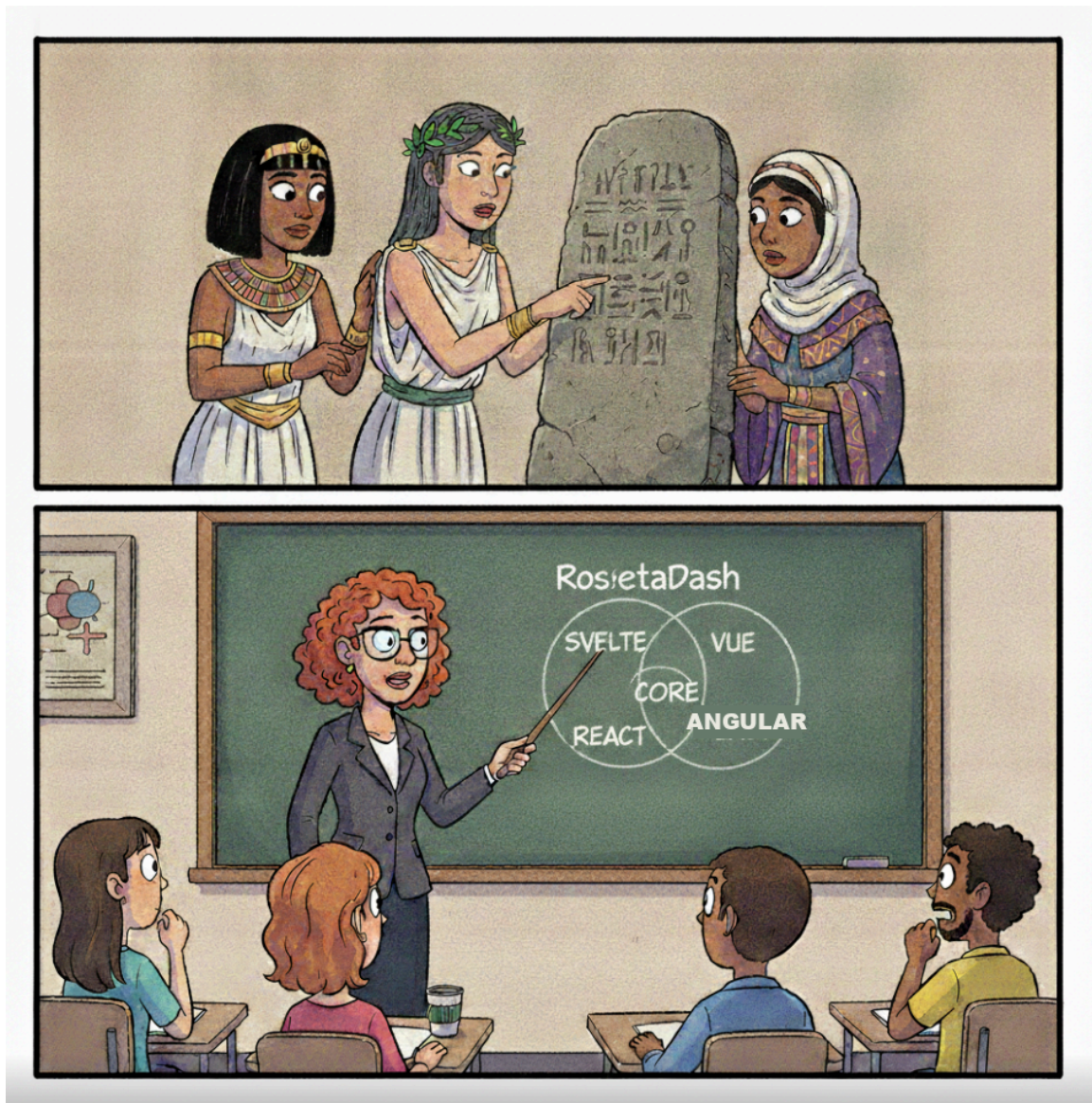


Figure 1: RosettaDash

These days, when AI helps us author our code, we are still forced to think in the context of one framework at a time. There is no front-end equivalent to Haxe, a high-level language

that compiles to Java, C#, Python, and even JavaScript. In that spirit, this project provides developers a single source of truth for their cross-framework component requirements.

RosettaDash runs locally as a component factory. You compose on a canvas and export real source files. The default zip is standalone: drop it into a project, set environment variables, and run. More often, you install only the npm modules you need — the full authoring environment with the builder, or a framework-specific library together with the shared cross-framework core.

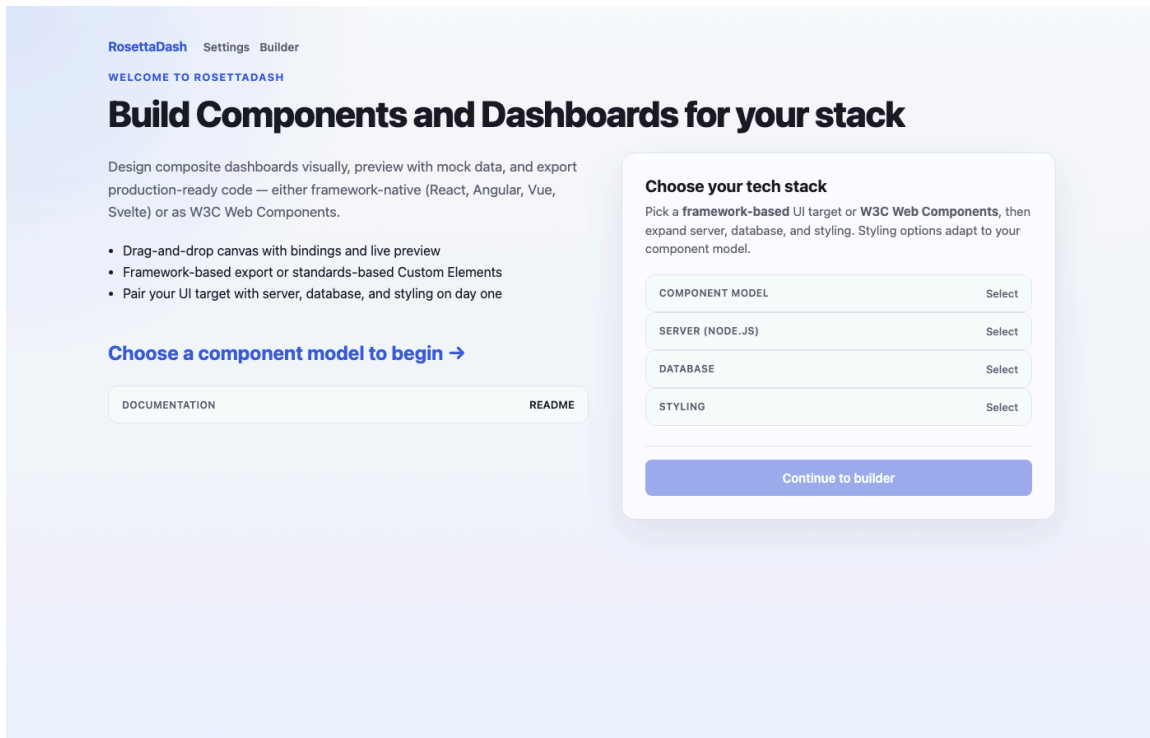


Figure 2: Factory still — the builder welcome, or a component that already left as a zip.

What “travel” means

A RosettaDash component can have a type, properties, ports, and events, and it can target whichever framework(s) that you need. The same definition can be exported as React, Angular, Vue, Svelte, or a W3C custom element. Server and database partners are available if you need them (Next, Nuxt, Nest, Express; Mongo, Postgres, Supabase, MySQL). You can also simply export the UI component and integrate it with an existing codebase.

All four server frameworks and four databases are not just export targets. Clone the repo and you can prove exports against real engines in Docker: `npm run parity:db:up`, then `parity:generate` and `parity:servers:up`. `npm run parity:check` confirms each generated server returns the same seeded rows. Idiomatic pairings: React→Next, Angular→Nest, Vue→Nuxt, Svelte→Express. Article 6 walks through the live Stack demo.

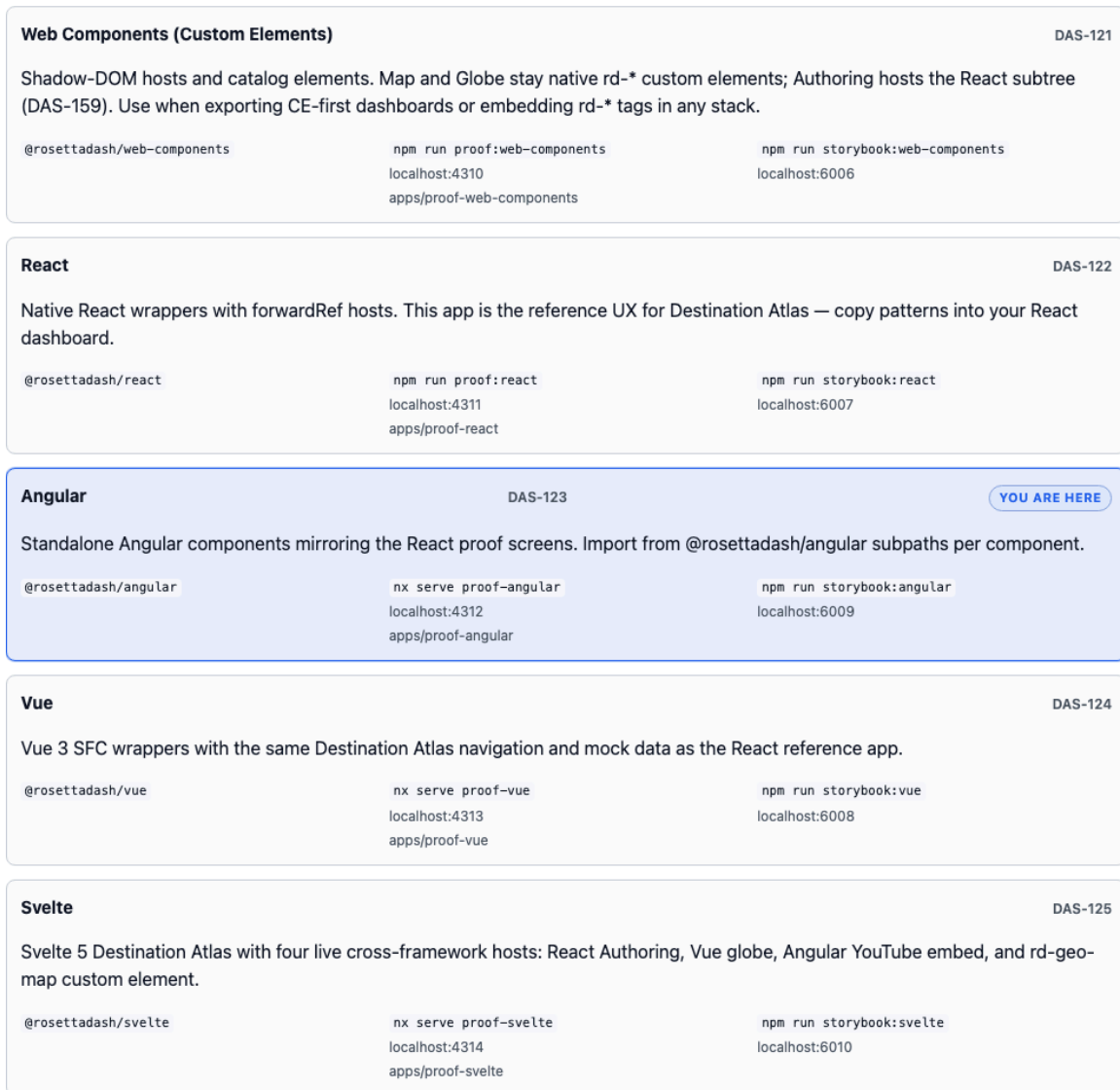


Figure 3: One contract, five spoken languages — a single component shown as React, Angular, Vue, Svelte, and a custom element.

Four ways in

The builder is the main authoring surface, for developers who want to get close to the code. Run `npm start`, open localhost:4200, pick a stack, compose, preview, and export. The next article covers it in detail.

Storybook is available for each framework, on ports 6006 through 6010. You may just need to see how a single piece works, how it is affected by parameter changes. Storybook can be a real time-saver. This is reviewed in article 3.

Destination Atlas is a single app built to show every component working together. The five proof apps share one dataset, and the app is discussed in articles 4 through 6.

npm is used to provide developers a simple way to include prebuilt components for individual frameworks: `@rosettadash/react`, `@rosettadash/angular`, `@rosettadash/vue`, `@rosettadash/svelte`, or `@rosettadash/web-components`. Use the framework-based version you need when you import a typed component into an existing app.

Runtimes — proof apps & Storybook

Each runtime ships a **proof app** (full Destination Atlas UX) and a **Storybook catalog** (isolated component review). Use the same npm package in your consumer project.

The Web Components, React, Angular, Vue, and Svelte blocks below are not RosettaDash palette components — they are npm runtime packages (delivery targets). Each card is proof-app documentation UI: plain layout markup styled for this About page. The actual reusable components live inside those packages (KpiCard, GeoMap, ScrollRegion, etc.) and appear on the other tabs and in Storybook.

PACKAGE	PROOF APP	STORYBOOK
Web Components (Custom Elements) DAS-121		
Shadow-DOM hosts and catalog elements. Map and Globe stay native rd-* custom elements; Authoring hosts the React subtree (DAS-159). Use when exporting CE-first dashboards or embedding rd-* tags in any stack.		
<code>@rosettadash/web-components</code>	<code>npm run proof:web-components</code> localhost:4310 apps/proof-web-components	<code>npm run storybook:web-components</code> localhost:6006
React DAS-122		
Native React wrappers with forwardRef hosts. This app is the reference UX for Destination Atlas — copy patterns into your React dashboard.		
<code>@rosettadash/react</code>	<code>npm run proof:react</code> localhost:4311 apps/proof-react	<code>npm run storybook:react</code> localhost:6007
Angular DAS-123 YOU ARE HERE		
Standalone Angular components mirroring the React proof screens. Import from <code>@rosettadash/angular</code> subpaths per component.		
<code>@rosettadash/angular</code>	<code>nx serve proof-angular</code> localhost:4312 apps/proof-angular	<code>npm run storybook:angular</code> localhost:6009
Vue DAS-124		
Vue 3 SFC wrappers with the same Destination Atlas navigation and mock data as the React reference app.		
<code>@rosettadash/vue</code>	<code>nx serve proof-vue</code> localhost:4313 apps/proof-vue	<code>npm run storybook:vue</code> localhost:6008
Svelte DAS-125		
Svelte 5 Destination Atlas with four live cross-framework hosts: React Authoring, Vue globe, Angular YouTube embed, and rd-geo-map custom element.		
<code>@rosettadash/svelte</code>	<code>nx serve proof-svelte</code> localhost:4314 apps/proof-svelte	<code>npm run storybook:svelte</code> localhost:6010

Figure 4: The four doors — builder, Storybook, Destination Atlas, npm

Eleven page-embed widget demos ship with the repo — a React weather card, a Vue flight board, an Angular stock ticker, a Svelte transit list, a custom-element 360° tour, and others. They use the same components as the builder, without the Destination Atlas shell or the builder canvas. To preview one on a host page before opening a proof app, `npm run demo:weather` on port 4320 is enough. Look to the end of article 7 or the `package.json` for the full set of demo apps.

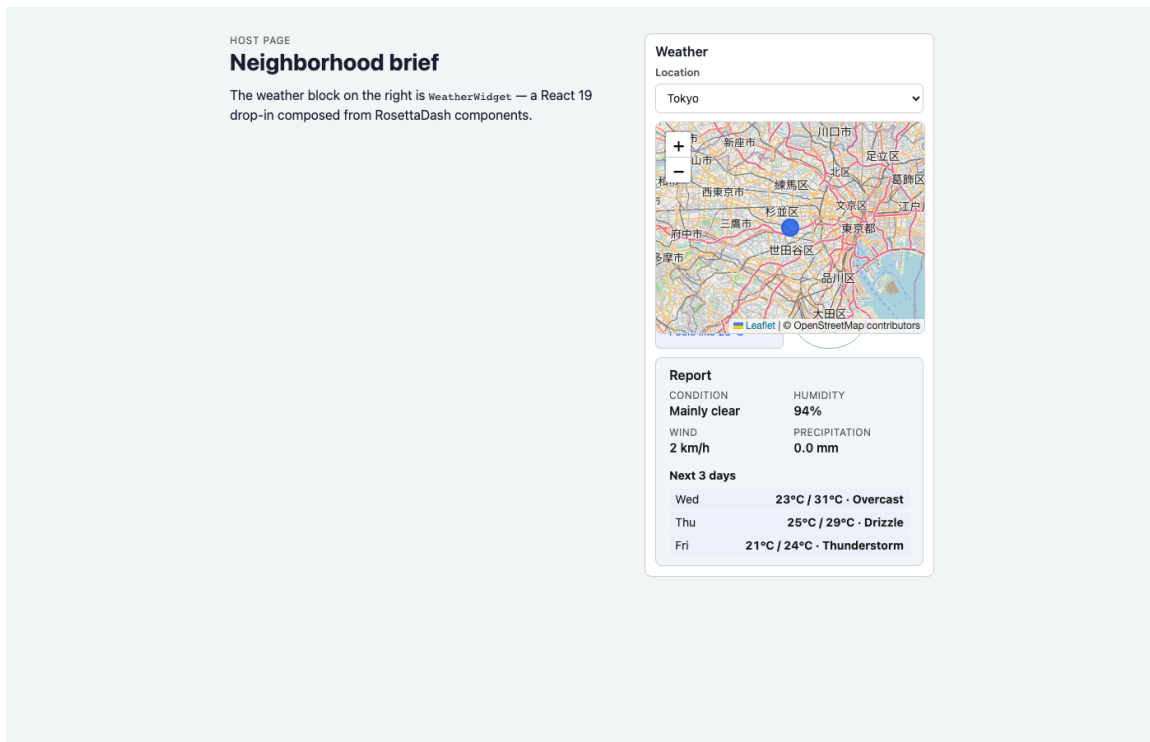


Figure 5: One demo widget on a host page — weather, or the 360 tour. Proof that a composite can leave without Atlas.

RosettaDash is aimed at teams that ship production software — full-stack developers, frontend specialists who need exports that match their repo conventions, and agencies working across more than one client stack.

Next

The next article covers the builder: palette, canvas, inspector, templates, preview, and export.